

Mini Hiring Pipeline

Design and architecture summary · Praviveek

GitHub: <https://github.com/21pravi/mini-hiring-pipeline>

Live demo: <https://mini-hiring-pipeline.onrender.com>

A web app for a recruiter running one job. Candidates move Applied → Screening → Interview → Offer → Hired, one stage at a time, or are rejected before Hired. Every move is appended to an event log that the database itself refuses to change, so the history is a real audit trail. One search box takes names (typo-tolerant) and short filters, combines them, ranks the results, and tells the recruiter why a query can't work instead of showing an empty list.

Stack	Python 3.10+, FastAPI, SQLite (stdlib), plain HTML/CSS/JS. No frontend build step.
Run	<code>pip install -r requirements.txt</code> → <code>python -m scripts.seed --reset</code> → <code>uvicorn app.main:app</code> → <code>localhost:8000</code>
Tests	105 pytest tests: state machine, audit trail (including raw SQL tampering), query parser, search ranking and explanations, HTTP API.
Repo also has	README (run, decisions, next steps), ai-logs/, screenshots.

Senior Data Analyst

25 candidates, 17 still in the pipeline, 4 waiting over a week

Add candidate

Find Priya Sharma, typed with a typo
sharam

In Interview right now
in:interview

Stuck in Screening over a week
in:screening for:>1w

Moved to Interview since Monday
moved:interview since:monday

Reached Offer, not hired
reached:offer is:rejected

Everyone except rejected
-is:rejected

► How search works

Applied 4

Aditya Mohanty
aditya.mohanty@example.com
5d in stage —
Move to Screening
Reject

Pooja Sharma
pooja.sharma@example.com
3d in stage —
Move to Screening
Reject

Rohan Gupta
rohan.gupta@example.com
2d in stage —
Move to Screening
Reject

Tanvi Kulkarni
tanvi.k@example.com
—

Reject

Screening 7

Ishita Banerjee
ishita.b@example.com
14d in stage —
Move to Interview
Reject

Priyanka Sharma
priyanka.s@example.com
11d in stage —
Move to Interview
Reject

Ananya Iyer
ananya.iyer@example.com
8d in stage —
Move to Interview
Reject

Harsh Agarwal
harsh.agarwal@example.com
—

Reject

Interview 4

Siddharth Joshi
sid.joshi@example.com
3d in stage —
Move to Offer Reject

Priya Sharma
priya.sharma@example.com
2d in stage —
Move to Offer Reject

Lakshmi Menon
lakshmi.menon@example.com
1d in stage —
Move to Offer Reject

Vikram Rao
vikram.rao@example.com
7h in stage —
Move to Offer Reject

Offer 2

Priya Verma
priya.verma@example.com
5d in stage —
Mark hired Reject

Sneha Patel
sneha.patel@example.com
1d in stage —
Mark hired Reject

Hired 2

Arjun Mehta
arjun.mehta@example.com
6d ago

Aman Verma
aman.verma@example.com
12h ago

Rejected 6

Daniel Fernandes
daniel.f@example.com
17d ago

Rahul Nair
rahul.nair@example.com
14d ago

Fatima Khan
fatima.khan@example.com
11d ago

Neha Kapoor
neha.kapoor@example.com
4d ago

Meera Das
meera.das@example.com
2d ago

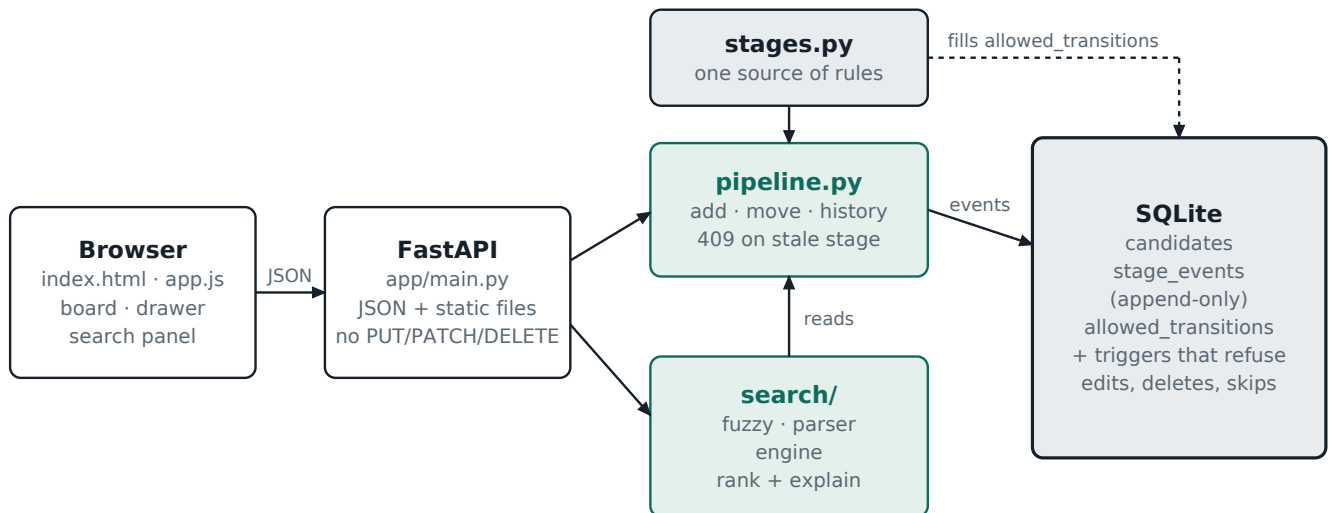
Nikhil Sharma
—
—

The board. Columns by stage, longest-waiting first; amber marks anyone stuck over a week. Buttons offer only the legal next move.

Mini Hiring Pipeline · Praviveek

1

Architecture



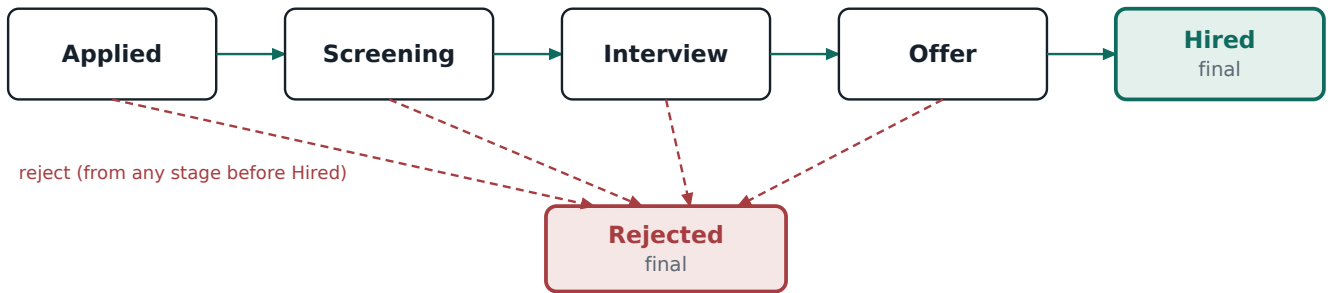
Three layers, one rule file. The browser talks JSON to a thin FastAPI layer. pipeline.py owns writes; search/ reads a projection built by pipeline.py. SQLite holds the event log and enforces the rules again with triggers.

Module	Responsibility
stages.py	Stages and every legal (from, action, to). Used by the pipeline, the DB triggers and the query parser, so the process is defined in one place.
db.py	Schema and triggers. Opens writes with BEGIN IMMEDIATE so read-check-write is serialised.
pipeline.py	Add candidate (name/email validation), move candidate, build history with time spent in each stage. Current stage = last event.
search/fuzzy.py	Accent folding, optimal string alignment distance (counts a transposition as one typo), prefix and typo scoring for name tokens.
search/parser.py	Query language → clauses. Errors carry a message, a hint, the character span and, when obvious, a corrected query.
search/engine.py	Filter, rank, attach reasons ("sharam" ≈ Sharma, 1 typo), explain empty results.
main.py	Routes, error mapping (409 stale, 422 illegal, 400 bad query), static files.
static/	Board, candidate drawer with live time-in-stage, search panel, dialogs.

A move, end to end

The board sends **POST /api/candidates/{id}/transitions** with the action and the stage the recruiter was looking at. The pipeline takes the write lock, loads the candidate, and answers 409 if the stage has changed since (double-click, second tab) or 422 if the outcome is already final. Otherwise it inserts one event. The insert trigger re-checks the sequence number, the from-stage, that the move is listed in **allowed_transitions**, and that the timestamp isn't earlier than the last event. The response is the updated candidate and the board re-renders.

Data model and audit trail



The state machine. Teal: advance one stage. Dashed: reject, allowed from any stage before Hired. Hired and Rejected are final.

Table	Columns	Rules
<code>candidates</code>	id, name, email (unique), created_at	No UPDATE, no DELETE.
<code>stage_events</code>	id, candidate_id, seq, action, from_stage, to_stage, at, note (≤ 500 chars)	Append-only. UNIQUE(candidate_id, seq). Insert trigger validates the chain.
<code>allowed_transitions</code>	from_stage, action, to_stage	Filled from stages.py on first run. Read-only.

There is no stage column on candidates. The current stage is the latest event, so the board and the history can't disagree, and time in stage is now minus that event's timestamp.

Guarantees and where they are enforced

Requirement	API / pipeline	Database	Tested by
History never changes	No PUT/PATCH/DELETE routes	Triggers abort UPDATE and DELETE	Raw SQL update/delete fails
One stage at a time	Only next_stage() is offered	Move must be in allowed_transitions	Raw insert skipping a stage fails
No reversing a final outcome	422 on Hired/Rejected	No transitions out of final stages	API + raw SQL
No lost or doubled moves	expected_stage → 409 if stale	seq must be max+1; from_stage must match	Double-submit test
Honest timeline	at = max(now, last event)	at can't precede the previous event	Backdating insert fails

Karthik Reddy
karthik.reddy@example.com

Close

Rejected 1 minute ago
Since Sat, Sep 26, 2026, 09:36 PM

Applied Screening Interview Offer Hired

History

Applied
Thu, Sep 17, 2026, 09:36 PM, stayed 5 days

Moved to Screening
Tue, Sep 22, 2026, 09:36 PM, stayed 4 days

Rejected at Screening
Sat, Sep 26, 2026, 09:36 PM
Went with a candidate with more dbt experience

History is permanent. Entries can't be edited or removed.

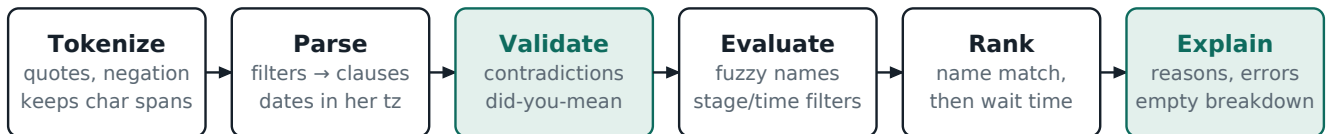
The history view

Opening a candidate shows the current stage and a live “time in stage” (it ticks while open), a step track with the stage where a rejected candidate stopped, and every event with its date and how long they stayed. Reasons given for Hired or Rejected are stored on the event.

The drawer says plainly that history is permanent. There is no edit button because there is nothing in the API or the database that could carry out an edit.

Search

A small query language, like Gmail's search box. Free text is a name; **key:value** terms are filters; everything is ANDed; - negates a filter.



Recruiter's question	Query	Seed data result
Find Priya Sharma, typed "sharam"	sharam / priya sharam	All four Sharmas / Priya Sharma first
Who's in Interview right now?	in:interview	4 people
Stuck in Screening over a week	in:screening for:>1w	4, longest first
Moved to Interview since Monday	moved:interview since:monday	5 (Monday in her time zone)
Reached Offer, not hired	reached:offer is:rejected	Neha Kapoor, Meera Das
Everyone except rejected	-is:rejected	19 of 25
Combined	priya -is:rejected	Priya Verma, Priya Sharma, Priyanka Sharma

Ranking

Name match score first (exact 1.0, prefix, then one or two typos; allowed typos scale with word length). Ties, and filter-only queries, are ordered by time in current stage, longest first, because the person waiting longest is the one the recruiter most likely needs. Each result shows why it matched.

Two kinds of "nothing"

Input	Kind	What she sees
in:interview	Error	"interview" isn't a stage. Did you mean in:interview? One-click fix.
in:hired in:offer	Error	Someone can only be in one stage at a time; use a comma for either.
reached:hired is:rejected	Error	Nobody in Rejected can have reached Hired.
since:today before:yesterday	Error	The window is empty (dates shown).
who is in interview right now	Empty	Reads like a question; suggests in:interview.
priyanka in:offer	Empty	Each part matches someone (1 and 2) but no one matches both.

Search in the app

×

Find Priya Sharma, typed with a typo
sharam

In Interview right now
in:interview

Stuck in Screening over a week
in:screening for:>1w

Moved to Interview since Monday
moved:interview since:monday

Reached Offer, not hired
reached:offer is:rejected

Everyone except rejected
-is:rejected

► How search works

Showing candidates where

2 matches best name match first

1 **Priya Sharma**
88% name match

Interview 2d in stage

Name: "sharam" ≈ Sharma (1 typo)

2 **Priyanka Sharma**
79% name match

Screening 11d in stage

Name: "sharam" ≈ Sharma (1 typo)

"priya sharam": Priya Sharma ranks first, each result says why it matched.

×

Find Priya Sharma, typed with a typo
sharam

In Interview right now
in:interview

Stuck in Screening over a week
in:screening for:>1w

Moved to Interview since Monday
moved:interview since:monday

Reached Offer, not hired
reached:offer is:rejected

Everyone except rejected
-is:rejected

► How search works

"interview" isn't a stage.

sharma in:interview

Did you mean in:interview? Stages are applied, screening, interview, offer, hired, rejected (or active, closed).

A mistyped stage: the bad part is underlined, the hint lists valid stages, and the button runs the corrected query.

×

Find Priya Sharma, typed with a typo
sharam

In Interview right now
in:interview

Stuck in Screening over a week
in:screening for:>1w

Moved to Interview since Monday
moved:interview since:monday

Reached Offer, not hired
reached:offer is:rejected

Everyone except rejected
-is:rejected

► How search works

Showing candidates where and

No matches

No one matches "In current stage for more than 3 weeks", so the combination can't match anyone either.

- Current stage is Offer: 2 people
- In current stage for more than 3 weeks: **nobody**

A valid query that matches nobody: how many people each filter matches on its own, so she sees which one emptied the list.

Decisions and trade-offs

Decision	Why	Cost
Event log as source of truth	Stage and history can't drift apart; time in stage is free.	Current stage needs a lookup of the last event (cheap at this size).
Rules enforced in DB triggers too	A bug in a future endpoint, or someone with the DB file, still can't rewrite history.	Rules exist in SQL as well as Python; both read the same transition table.
expected_stage on every move	A double-click or second tab can't skip a stage.	Client must send the stage it saw; stale clients get a 409 and refresh.
Confirm only final outcomes	Hired/Rejected can't be undone; ordinary advances stay one click.	A mis-click forward is permanent until correction events exist.
Query language, not an LLM	Predictable, testable, errors point at the exact characters, and the app echoes what it understood.	She learns a few filters. Mitigated by example chips, syntax help and suggestions.
Fuzzy names, strict vocabulary	Names are unknown data; stage names are a closed set where a guess would hide a mistake.	"interview" is an error (with a fix) rather than silently corrected.
Search in Python, in memory	One job is hundreds of rows. Matching, ranking and explanations stay in one readable module.	Won't scale to many jobs without compiling to SQL.
Browser time zone for dates	"Since Monday" means her Monday, not UTC's.	One more parameter on the search request.
No drag-and-drop	Dropping a card two columns over is the easiest way to break the rules.	Less tactile board.

With more time

Correction events — record a mistaken move as a new event pointing at the old one.

Actor on every event — auth, and who made each move.

Tamper evidence — hash-chain events so edits made outside the app are detectable.

Natural-language front end — an LLM translates the question into the query language; she confirms the chips; the parser still runs it.

Autocomplete — for filter keys, stages and names.

Scale search — compile clauses to SQL, SQLite FTS5 trigram index for names.

Richer queries — OR across filters, parentheses, saved searches, per-stage stuck thresholds.

Multiple jobs, E2E tests (Playwright), CI, accessibility pass.